

Suite de Tests JobbingTrack

[← Développement](#) | [← Documentation](#) | [← README principal](#) |  [Navigation](#)

Suite complète de tests pour la plateforme JobbingTrack, incluant tests unitaires, d'intégration, E2E, de performance et de sécurité.

Structure des Tests

```
tests/
├── README.md                # Documentation complète (racine du projet)
├── package.json             # Configuration npm des tests
├── jest.config.js           # Configuration Jest
├── jest.setup.js            # Setup global Jest
├── playwright.config.ts     # Configuration Playwright
├── run-tests.js             # Script principal d'exécution
├── setup.js                 # Script de configuration
├── verify.js                # Script de vérification
├── cleanup.sh               # Script de nettoyage
├── docker-compose.test.yml  # Services de test
├── .env.test                # Variables d'environnement
├── .gitignore               # Fichiers à ignorer
├── docker/                  # Tests Docker et déploiement
│   ├── test-docker-images.js # Tests des noms d'images Docker
│   └── test-make-down-clean.js # Tests de la commande make down
├── system/                  # Tests système et vérification
│   └── verify-test-system.js  # Vérification complète du système
├── coverage/                # Rapports de couverture
├── temp/                    # Fichiers temporaires
├── results/                 # Résultats de tests
├── screenshots/             # Captures d'écran de tests
├── videos/                  # Vidéos de tests
├── fixtures/                # Données de test
│   ├── users.json           # Utilisateurs de test
│   ├── companies.json       # Entreprises de test
│   └── applications.json     # Candidatures de test
├── unit/                    # Tests unitaires
│   ├── test-utils.js         # Tests des utilitaires
│   ├── test-environment-variables.js
│   └── test-*.js             # Autres tests unitaires
├── integration/             # Tests d'intégration
│   ├── test-frontend-integration.js
│   └── test-full-system.js
```

```

|   └─ test-hydration-fixes.js # Tests des corrections d'hydratation
|   └─ test-implementation.js # Tests de l'implémentation complète
|   └─ test-websocket.js
|   └─ test-*.js
└─ database/ # Tests de base de données
    └─ test-database.js # Tests DB complets
    └─ test-postgresql-config.js
    └─ test-*.js
└─ api/ # Tests API
    └─ test-api.js # Tests API complets
    └─ test-*.js
└─ backend/ # Tests backend
    └─ test-services.js # Tests des services backend
    └─ test-*.js
└─ frontend/ # Tests frontend
    └─ test-frontend-improvements.js
    └─ test-login-improvements.js
    └─ test-runtime-error-fixes.js
    └─ test-theme-system.js
    └─ test-*.js
└─ mobile/ # Tests mobile
    └─ test-mobile.js # Tests mobile complets
    └─ test-*.js
└─ e2e/ # Tests End-to-End (Playwright)
    └─ specs/ # Spécifications de tests
        └─ admin-backoffice.spec.ts
        └─ user-journeys.spec.ts
        └─ *.spec.ts
    └─ fixtures/ # Données de test E2E
    └─ utils/ # Utilitaires E2E
    └─ results/ # Résultats E2E
└─ performance/ # Tests de performance
    └─ test-performance.js
    └─ test-*.js
└─ security/ # Tests de sécurité
    └─ test-security.js # Tests de sécurité complets
    └─ test-secure-env-vars.js # Tests sécurité des variables d'environnement

```

Scripts Principaux

run-tests.js

Script principal d'orchestration des tests

- Orchestre tous les types de tests (unit, integration, e2e, etc.)

- Exécution en séquence ou parallèle selon les options
- Génération de rapports automatiques
- Gestion des erreurs et timeouts

```
# Tous les tests sauf E2E
node run-tests.js

# Tous les tests incluant E2E
node run-tests.js --e2e

# Tests spécifiques
node run-tests.js --no-database --no-frontend
```

setup.js

Script de configuration initiale

- Vérification des prérequis (Node.js, Docker, etc.)
- Installation des dépendances
- Création des répertoires nécessaires
- Configuration des fixtures et variables d'environnement
- Génération de la documentation

```
# Configuration complète
node setup.js

# Ou via Makefile
make test-setup
```

verify.js

Script de vérification de la configuration

- Vérification de tous les fichiers de configuration
- Tests des dépendances et scripts npm
- Validation de la structure des dossiers
- Vérification des permissions et accès

```
# Vérification complète
node verify.js

# Ou via Makefile
make test-verify
```

Utilisation Rapide

Configuration Initiale

```
# 1. Configuration automatique
make test-setup

# 2. Vérification de la configuration
make test-verify

# 3. Démarrage des services de test
docker-compose -f tests/docker-compose.test.yml up -d
```

Exécution des Tests

Tests par Catégorie

```
# Tests unitaires
make test-unit

# Tests d'intégration
make test-integration

# Tests de base de données
make test-database

# Tests API
make test-api

# Tests backend
make test-backend

# Tests frontend
make test-frontend

# Tests mobile
make test-mobile

# Tests E2E (Playwright)
make test-e2e

# Tests E2E avec interface
make test-e2e-ui

# Tests de performance
make test-performance
```

```
# Tests de sécurité  
make test-security
```

Tests Spécialisés (Réorganisés)

```
# Tests Docker et déploiement  
make test-docker-images      # Tests des images Docker  
make test-docker-clean      # Tests de la commande make down  
  
# Tests système et vérification  
make test-system-verify      # Vérification complète du système  
  
# Tests d'intégration étendus  
make test-hydration          # Tests des corrections d'hydratation  
make test-implementation     # Tests de l'implémentation complète  
  
# Tests de sécurité des variables d'environnement  
make test-secure-env         # Tests sécurité variables d'environnement
```

Tests Complets

```
# Suite complète (tous types)  
make test-all  
  
# Tests rapides (sans E2E)  
make test-quick  
  
# Tests backend uniquement  
make test-backend-only  
  
# Tests frontend uniquement  
make test-frontend-only  
  
# Tests avec coverage  
make test-coverage  
  
# Tests avec rapport  
make test-report
```



Rapports et Coverage

Génération Automatique

-  Rapports HTML Playwright

-  Rapports JSON pour CI/CD
-  Rapports JUnit pour intégration
-  Coverage Jest avec détails
-  Rapports de performance
-  Rapports de sécurité

Emplacement des Rapports

```
tests/reports/
├─ test-report.json          # Rapport principal
├─ performance-report.json  # Performance
├─ security-report.json     # Sécurité
├─ playwright-report/      # E2E HTML
├─ junit-results.xml        # CI/CD
└─ coverage/               # Coverage détaillé

tests/coverage/             # Rapports de couverture Jest
tests/results/              # Résultats de tests divers
tests/screenshots/          # Captures d'écran de tests
tests/videos/               # Vidéos de tests E2E
tests/temp/                 # Fichiers temporaires
```

Données de Test (Fixtures)

Utilisateurs de Test

```
{
  "admin": {
    "email": "admin@jobbingtrack.com",
    "password": "admin123",
    "role": "admin"
  },
  "user": {
    "email": "user@jobbingtrack.com",
    "password": "user123",
    "role": "user"
  },
  "candidate": {
    "email": "candidate@jobbingtrack.com",
    "password": "candidate123",
    "role": "candidate"
  }
}
```

Entreprises de Test

```
{
  "techCorp": {
    "name": "Tech Corp",
    "industry": "Technology",
    "size": "50-200"
  },
  "startup": {
    "name": "Startup Inc",
    "industry": "Technology",
    "size": "1-10"
  }
}
```

Configuration

Variables d'Environnement

```
# Copier et adapter
cp tests/.env.test tests/.env.test.local

# Variables principales
NODE_ENV=test
DATABASE_URL=postgresql://jobbingtrack:jobbingtrack123@localhost:5432/jobbingt
API_GATEWAY_URL=http://localhost:3000
TEST_TIMEOUT=30000
```

Configuration Jest

- Test environment: Node.js
- Coverage activ 
- Timeout: 10s par d faut
- Setup global: tests/jest.setup.js

Configuration Playwright

- Base URL: <http://localhost:8080>
- Navigateurs: Chrome, Firefox, Safari, Mobile
- Timeout: 30s
- Screenshots en cas d  chec
- Vid  os pour tests E2E

Dépannage

Tests échouent

```
# Vérifier les services
make status

# Vérifier les logs
make logs

# Redémarrer les services
make down && make up

# Nettoyer et redémarrer
make test-clean && make up
```

Tests E2E échouent

```
# Vérifier que les services sont démarrés
make up

# Attendre que les services soient prêts
sleep 30

# Vérifier les URLs dans playwright.config.ts
# Attendre les éléments avant d'interagir
```

Tests de base de données échouent

```
# Vérifier la connexion DB
make db-status

# Vérifier les variables d'environnement
# Utiliser la bonne DATABASE_URL
```

Bonnes Pratiques

Tests Unitaires

-  Tester une fonction à la fois
-  Utiliser des mocks pour les dépendances externes
-  Tester les cas d'erreur
-  Noms de tests descriptifs

Tests E2E

-  Tests indépendants
-  Attendre les éléments avant d'interagir
-  Tests de données cohérentes
-  Cleanup après chaque test

Tests de Performance

-  Mesurer les métriques réelles
-  Tests de charge réalistes
-  Monitoring des ressources
-  Rapports détaillés

Tests de Sécurité

-  Tests des vulnérabilités courantes
-  Validation des entrées
-  Tests d'authentification
-  Tests d'autorisation



Navigation et Références



Documentation Complémentaire

-  [Tests Racine](#) - Documentation complète du dossier tests/
-  [Stratégie de Tests](#) - Méthodologie de tests
-  [Guide de Développement](#) - Configuration environnement
-  [Guide Frontend](#) - Tests frontend spécifiques
-  [Guide de l'API](#) - Tests API



Interface Playwright

-  Interface Admin Playwright - Interface graphique pour les tests E2E (si disponible dans le frontend)



Scripts et Outils

Scripts Principaux

-  run-tests.js - Orchestration de tous les tests
-  setup.js - Configuration initiale complète
-  verify.js - Vérification de la configuration

Scripts Spécialisés

-  test-docker-images.js - Tests des images Docker
-  test-hydration-fixes.js - Tests des corrections d'hydratation
-  test-implementation.js - Tests de l'implémentation complète
-  test-secure-env-vars.js - Tests sécurité des variables d'environnement

Scripts Utilitaires

-  cleanup.sh - Nettoyage de l'environnement de test
-  Génération de rapports automatiques



Commandes Makefile

```
# Tests par catégorie
make test-unit test-integration test-database test-api
make test-backend test-frontend test-mobile test-e2e
make test-performance test-security

# Tests spécialisés
make test-docker-images test-docker-clean
make test-system-verify test-hydration
make test-implementation test-secure-env

# Tests complets
make test-all test-quick test-coverage test-report
make test-setup test-verify test-clean
```



Flux de Travail Recommandé

1. **Configuration** : `make test-setup`
2. **Vérification** : `make test-verify`
3. **Développement** : Tests unitaires en continu
4. **Intégration** : Tests d'intégration avant commit
5. **Validation** : Tests E2E avant déploiement
6. **Performance** : Tests de charge avant release



Objectifs de Qualité

Coverage

- **Global:** > 80%
- **Services critiques:** > 90%
- **Composants frontend:** > 75%
- **Intégration:** > 70%

Métriques Performance

- **Temps réponse API:** < 200ms (p95)
- **Temps chargement page:** < 2s
- **Lighthouse score:** > 90
- **Core Web Vitals:** Tous "Good"

Standards Qualité

-  Zero erreur ESLint
-  Zero warning TypeScript
-  Tests passent à 100%
-  Coverage objectifs atteints
-  Pas de dépendances vulnérables
-  Code review approuvé



Ressources Supplémentaires

- [Configuration Développement](#) - Environnement de développement
 - [Workflow Développement](#) - Processus de développement
 - [Documentation Makefile](#) - Commandes automatisées
 - [API Reference](#) - Documentation API
 - [Guide Sécurité](#) - Bonnes pratiques sécurité
-



Suite de tests complète et prête à l'emploi !

Pour commencer : `make test-setup` puis `make test-all` 

Version: 4.1

Dernière mise à jour: Octobre 2025